جامعة الفيصل
Alfaisal University

كلية الهندسة والحوسبة المتقدمة
College of Engineering and Advanced Computing

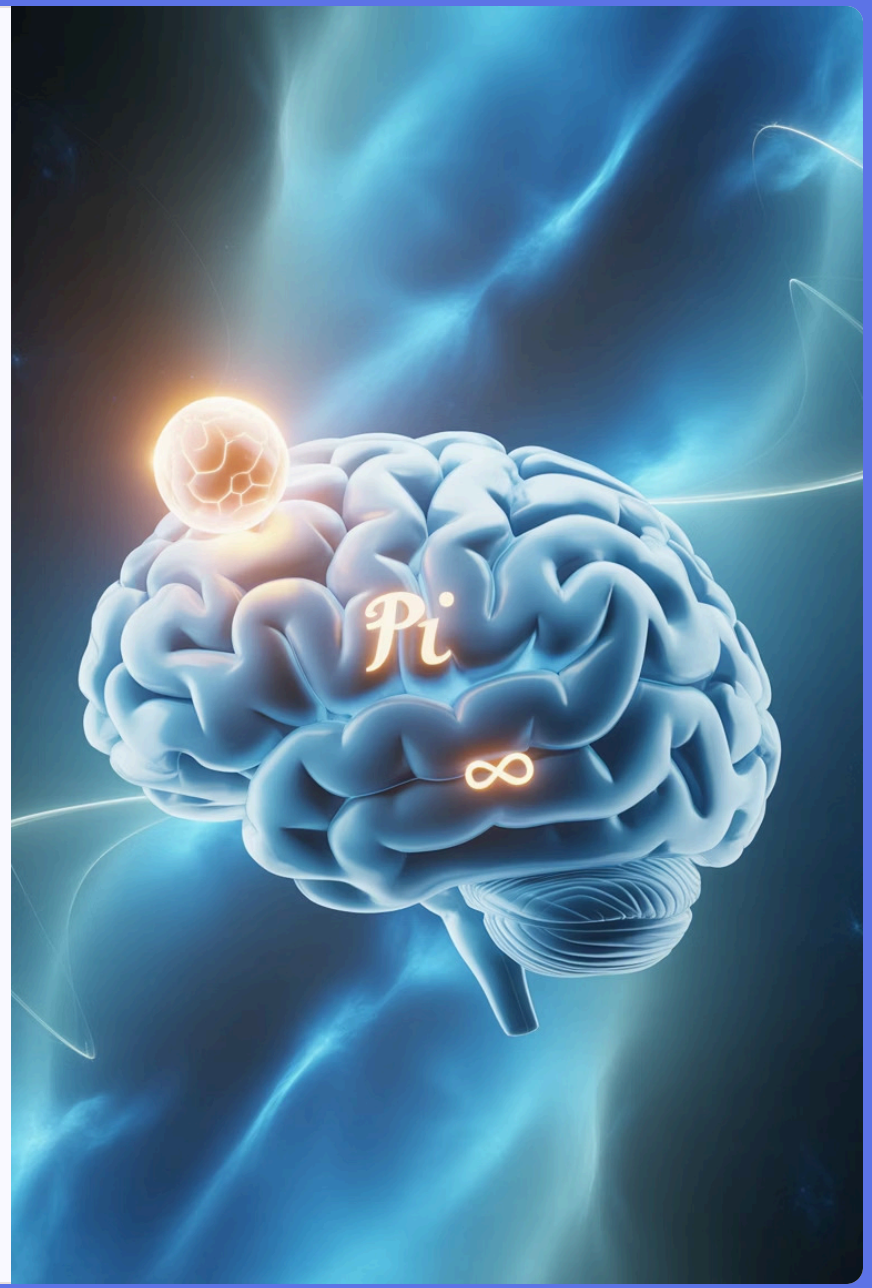
Introduction to Artificial Intelligence (SE 444)

Lecture 11: Bayesian Networks & Probabilistic Inference [Part 1]

Prof. Anis Koubaa

akoubaa@alfaisal.edu

This lecture introduces **Bayesian Networks** as powerful probabilistic graphical models, essential for managing uncertainty and complexity in AI. We explore their formal definition (DAGs + CPTs), key concepts like **conditional independence** and **d-separation**, and various inference methods from exact to approximate.



Why Probabilistic Models?

Real-world environments are fundamentally **uncertain**:

Noisy Sensors

Measurements contain errors and imprecision

Probabilistic Diagnoses

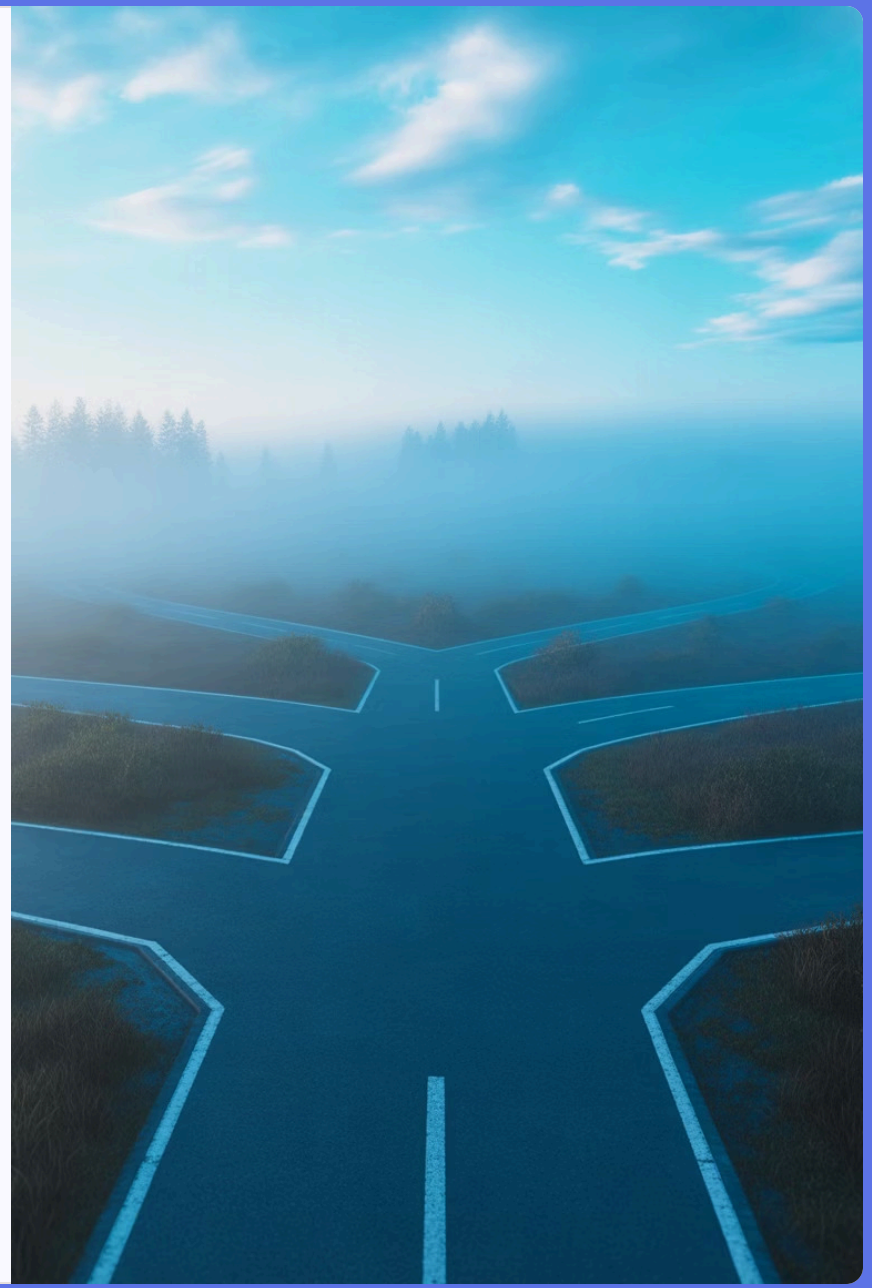
Medical outcomes are never guaranteed

Stochastic Actions

Outcomes vary despite identical inputs

Hidden Causes

Unobserved factors influence what we see



Why Bayesian Networks?

Real-world systems involve **uncertainty** and **complex interactions**.

A Bayesian network is a map that shows how events influence each other under uncertainty.

Why They Matter

Bayesian networks let us **connect the dots** between **causes** and **effects** and **reason** even when information is missing.

Where Do We Need Them?

- **Medical diagnosis**
  Infer: Does the patient have a disease given symptoms or test results?
- **Fault detection**
  Determine which component failed based on noisy sensor readings.
- **Weather forecasting**
  Predict rain given clouds, humidity, and wind.
- **Robotics perception**
  Estimate hidden world states from uncertain sensor inputs.

The Problem of Complexity

The Cost of Full Joint Probability Tables

If we model uncertainty with a **full joint distribution** over *n* binary variables:

- Total required parameters: $2^n - 1$
- Complexity grows *exponentially*
- Impossible to store, estimate, or compute for real systems

Why This Is a Problem

- Hard to understand the relationships
- Hard to update when domain knowledge changes
- Hard to use for real-time inference

Example

For just **20 binary variables**:

- Full joint table size:
  1,048,576 parameters
- Even small problems become computationally impossible

Takeaway

The real world is too complex for full joint distributions — we need structure.

Key Advantages of Bayesian Networks

Bayesian networks offer significant benefits for modeling and inference in uncertain domains.



Reduction of Complexity

Instead of massive combined cases, networks simplify problems into local, direct relationships, leading to faster inference and more efficient models.



Powerful Inference

They support both predictive (forward) and diagnostic (backward) reasoning, closely mirroring human cognitive processes for explaining and forecasting.



Causal Modeling

Bayesian networks explicitly capture causal links, clarifying why events occur and predicting the effects of interventions in complex, uncertain systems.

How Bayesian Networks Help

Bayesian Networks Reduce Complexity Through Structure

Bayesian networks assume that **each variable only depends on a few direct influences (parents)**.

This leads to:

- **Small local CPTs** instead of one huge table
- **Fewer parameters to estimate**
- **More efficient inference**
- **Modular, interpretable models**

Why This Is Powerful

We replace exponential growth with **linear or near-linear** growth in model size.

Burglary–Alarm Network

5 variables with intuitive causal relationships:

- Burglary
- Earthquake
- Alarm
- JohnCalls
- MaryCalls

Thanks to structure:

- **Only 10 parameters needed**
- Versus **31** in a traditional representation

The Problem with Classical Logic

The Exponential Challenge

Consider a domain with **n** binary variables.

A full joint probability distribution requires **O(2^n)** parameters. For just 30 variables, that's over 1 billion probabilities to specify and store!

- **Exponential storage** requirements
- **Exponential computation** for inference
- **Impossible to learn** from sparse data
- Does not align with **human reasoning** patterns

Bayesian Network's Solution

- **Exploit conditional independence**
- Use **graph structure** to factorize joint distributions
- Leverage **local conditional probability tables (CPTs)**
- Enables **efficient inference** algorithms

"Bayesian Networks are to probabilistic reasoning what propositional logic is to deterministic reasoning: a compact, modular representation that mirrors how humans naturally decompose complex problems."

The Problem with Classical Logic

Classical Logic

❌ Cannot encode **uncertainty**

❌ Binary truth values only

❌ No degrees of belief

Full Joint Distribution

❌ Exponentially large

$$P(X_1, \ldots, X_n) \text{ requires } O(2^n) \text{ parameters}$$

❌ Computationally intractable

We Need Compact Representations

Two fundamental requirements:

1. Compact Representation

Encode distributions efficiently without exponential parameters

2. Efficient Inference

Answer probabilistic queries in reasonable time

Bayesian Networks solve both problems elegantly.



🧠 Bayesian Networks: Formal Definition

A **BN** is a probabilistic graphical model that represents a set of **random variables** and their conditional **dependencies**.

🔷 Directed Acyclic Graph (DAG)

The structure: nodes represent **random variables**, and directed edges indicate **dependencies**.

📊 Conditional Probability Tables (CPTs)

The parameters: quantitative probabilities for each variable given its parents.

Together, these encode the full **joint probability distribution** in a compact, structured form.

🧮 Formal Notation

$$BN = (G, \Theta)$$

G: a **DAG** whose nodes are **random variables** and edges indicate **dependencies**.

Θ: a collection of **CPTs** defining $P(X|Parents(X)$ for every variable.

Bayesian Networks: Formal Definition

A **Bayesian Network** is a pair (G, Θ) where:

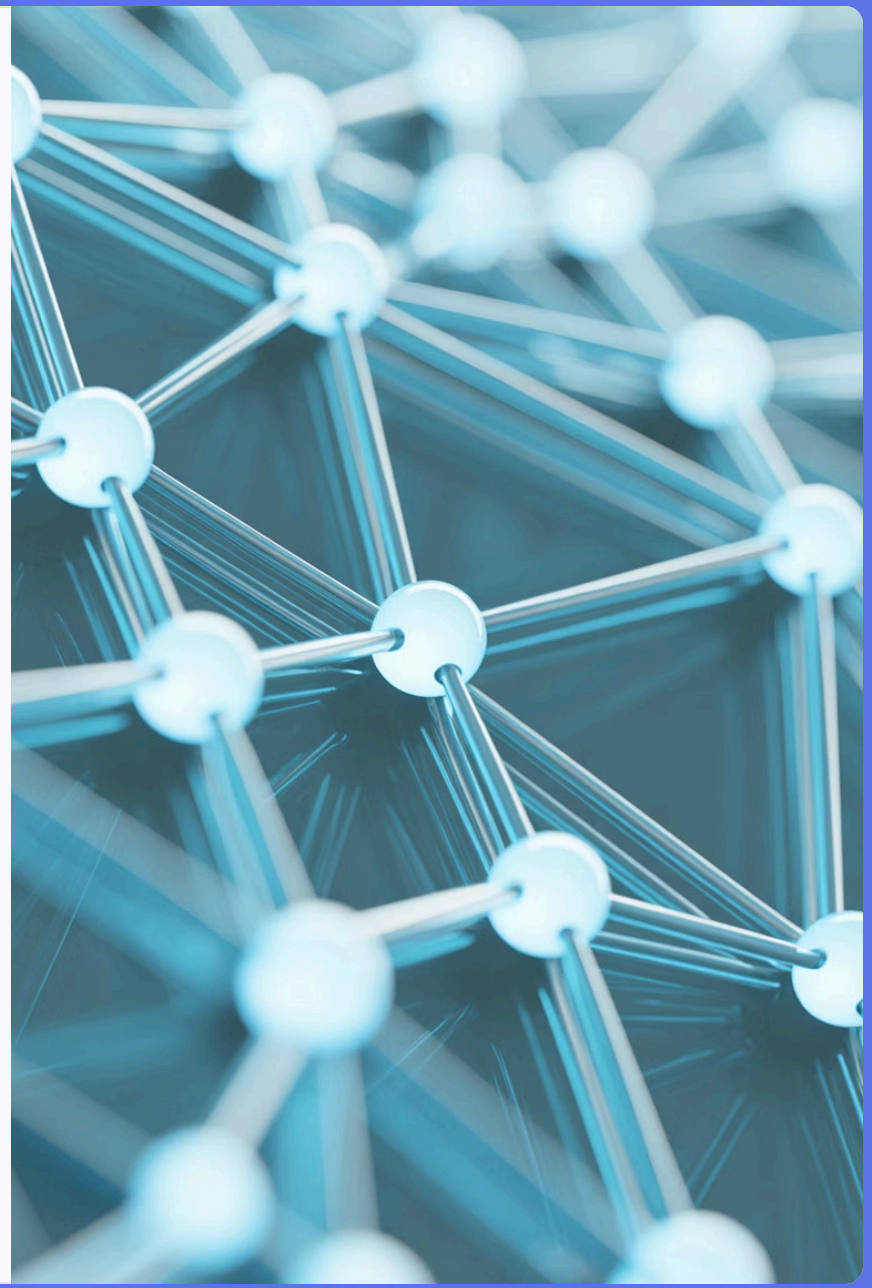
G: Directed Acyclic Graph (DAG)

- Nodes represent random variables
- Edges encode probabilistic dependencies
- No cycles allowed

Θ: Conditional Probability Tables

One CPT for each variable:

$$\theta_i = P(X_i \mid \text{Parents}(X_i))$$



Components of a Bayesian Network

1️⃣ Graph Structure (G)

- **Nodes = variables** (e.g., Disease, Fever)
- **Arrows = direct influences** between variables
- Must be **acyclic** 🔁🚫 (no directed cycles allowed)
- Captures the **qualitative** structure of the domain (causal or statistical)

2️⃣ Conditional Probability Tables (Θ)

Each node has a **CPT** describing:

$$\mathbf{P(X \mid Parents(X))}$$

- Specifies how likely the variable is under each parent configuration
- Captures the **quantitative** part of the model
- CPT rows must **sum to 1** ✔️

🧩 **Together, the DAG and CPTs fully define the model's behavior.**

Example: Simple Medical Diagnosis Network

Key Variables & Structure

- Disease (e.g., Flu)
- Fever
- Cough
- Test Result (e.g., Blood Test)

Dependencies (arrows) reflect causal links:

- Disease causes Fever
- Disease causes Cough
- (Fever, Cough) influence Test Result

Why This Works

This network elegantly encodes:

- Causal relationships (Disease predicts symptoms)
- Probabilistic dependencies (Test Result depends on symptoms)

Enabling both:

- **Diagnostic Reasoning:** Infer Disease from Fever and Cough.
- **Predictive Reasoning:** Predict Fever from Disease.

Valid vs. Invalid BN

✔️ Valid Bayesian Network (DAG)

Rain → Sprinkler → Grass Wet

- No cycles
- Correctly represents causal flow
- Supports clean factorization of probabilities

❌ Invalid Bayesian Network (Has Cycle)

A → B → C → A

- Contains a **directed cycle**
- Violates the definition of a BN
- Breaks **conditional independence** assumptions
- Makes factorization **impossible**

Key Rule

A Bayesian Network must always be a **Directed Acyclic Graph (DAG)** — cycles make reasoning inconsistent.

CPTs (Conditional Probability Tables)

Every **node in a Bayesian Network** quantifies its probabilistic behavior using a **Conditional Probability Table (CPT)**. 📊

Examples of **CPTs**:

- **P(Disease)** — For a **root node**, it's a prior probability.
- **P(Fever | Disease)** — For a **child node**, it depends on its parents.
- **P(Test | Fever, Cough)** — Can depend on **multiple parents**.

⚖️ Sum to 1.0

Each row in a **CPT** must sum to **1.0**, representing the full probability space for that condition.



📈 Size Grows with Parents

The size of a **CPT** grows **exponentially** with the number of **parents** a node has.

Example CPT (Fever | Disease)

Disease	P(Fever = T)	P(Fever = F)
True	0.80	0.20
False	0.10	0.90

Interpretation

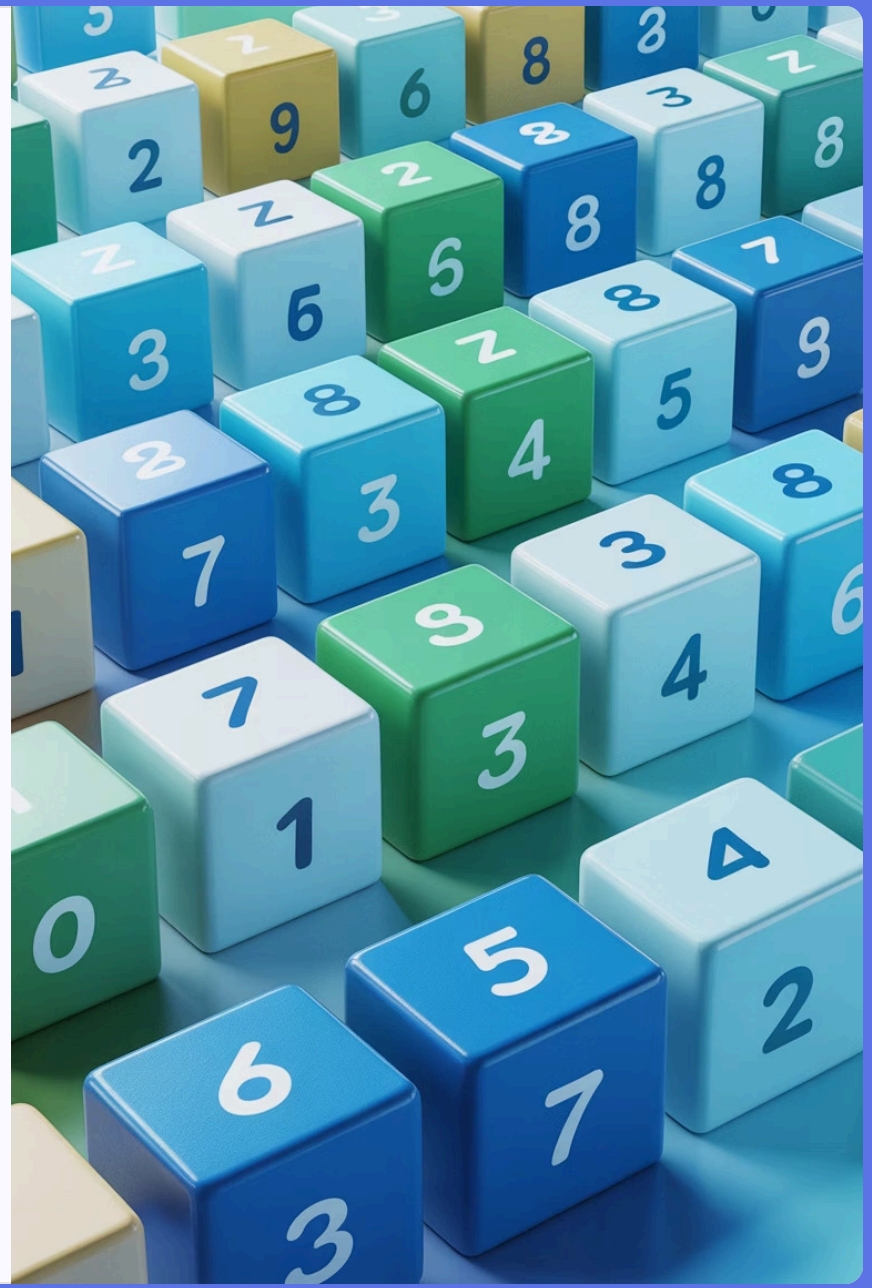
If a patient has the disease, fever is highly likely.

Key Property: Factorization

The joint distribution **factorizes** according to graph structure:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

This factorization eliminates **exponential redundancy** by encoding conditional independence relations through graph structure.



Worked Example: Factorization

Let's illustrate the factorization property with a simple example involving Rain, Sprinkler, and Grass Wetness.

$$P(R = T, S = F, G = T) = P(R = T) \times P(S = F \mid R = T) \times P(G = T \mid R = T, S = F)$$

Using hypothetical probabilities:

- P(Rain = True) = 0.2 (It rains 20% of the time)
- P(Sprinkler = False | Rain = True) = 0.9 (If it's raining, the sprinkler is off 90% of the time)
- P(Grass Wet = True | Rain = True, Sprinkler = False) = 0.95 (If it's raining and the sprinkler is off, the grass is still very likely wet from the rain)

We can compute the joint probability:

$$0.2 \times 0.9 \times 0.95 = 0.171$$

This demonstrates how the full joint probability of these three events can be computed efficiently by multiplying the relevant conditional probabilities, leveraging the compact structure of a Bayesian Network.

Example: Burglary Network Factorization

Classic example with 5 variables:

- **B**: Burglary occurred
- **E**: Earthquake occurred
- **A**: Alarm sounds
- **M**: Mary calls
- **J**: John calls

Burglary Network: Factorization

The joint probability factorizes as:

$$P(B, E, A, M, J) = P(B)P(E)P(A \mid B, E)P(M \mid A)P(J \mid A)$$

Instead of $2^5 = 32$ parameters, we need only **10 parameters**.

The graph structure makes independence explicit.

Conditional Independence 🤝

At the heart of probabilistic graphical models lies the concept of conditional independence. It helps us understand how variables relate to each other when we have additional information.

The essential semantic concept is represented by the notation:

$$X \perp Y \mid Z$$

This is read as: "**X** is independent of **Y** given **Z**."

*In plain language, it means that if we already know the state of **Z**, then knowing the state of **X** provides no additional information about the state of **Y**, and vice versa. Their direct relationship is "blocked" by **Z**.*

A simple scenario

X: Sprinkler

Is the sprinkler on?

Z: Rain

Is it raining?

Y: Wet Grass

Is the grass wet?

The sprinkler (**X**) and rain (**Z**) both make the grass wet (**Y**). What if you know for certain that it IS raining (**Z**)?

💡 Insight: Conditional Independence

If **Z** (rain) is known, then knowing **X** (sprinkler) tells you nothing new about **Y** (wet grass). Thus, **X** is independent of **Y** given **Z**.

This example demonstrates how graph structures in Bayesian networks represent **independence relationships**. The method to systematically identify these is called **d-separation**, which we'll explore next.



Three Canonical Structures

Understanding these patterns is crucial for **d-separation**:

1. Chain

$X \to Y \to Z$

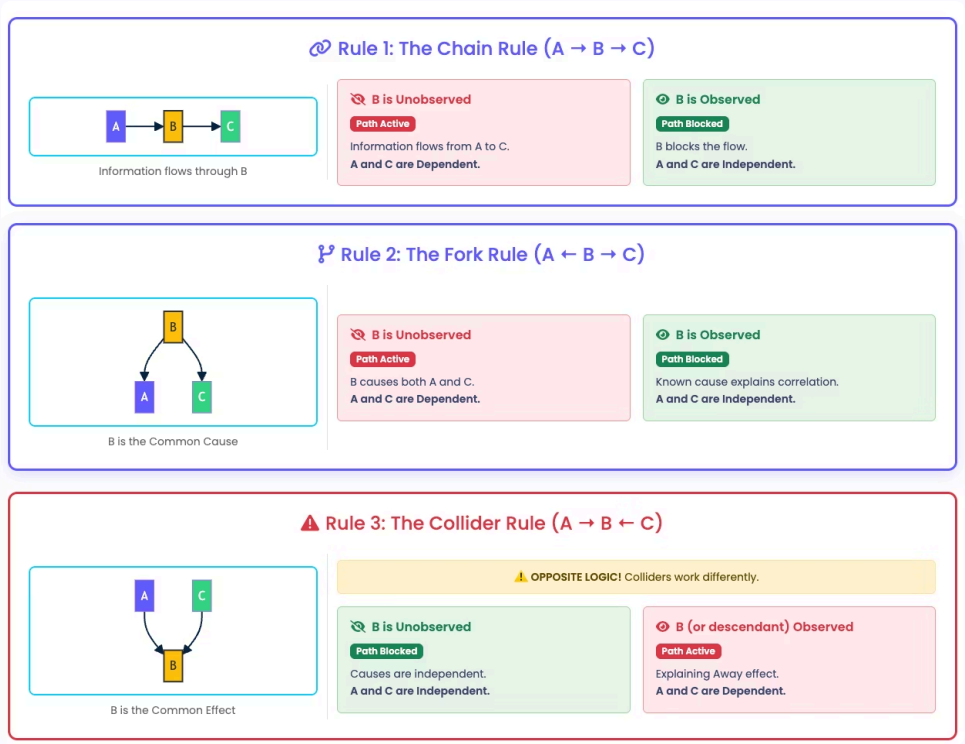
2. Fork

$X \leftarrow Y \to Z$

3. Collider

$X \to Y \leftarrow Z$

Three Canonical Structures



Understanding the 3 Basic Patterns

Chain: A → B → C

Like: **Weather → Traffic → Commute Time**

Rule: If the middle node (**B**) is observed, the path is blocked, rendering A and C conditionally independent.

Idea: If you know the Traffic conditions, knowing the Weather gives you no additional information about your Commute Time.

Fork: A ← B → C

Like: **Disease → Fever & Cough**

Rule: If the common parent (**B**) is observed, the path is blocked, making A and C conditionally independent.

Idea: Knowing the underlying Disease explains both symptoms (Fever and Cough), so they provide no extra information about each other.

Collider: A → B ← C

Like: **Burglary → Alarm ← Earthquake**

Rule: This is the opposite of Chain & Fork! If the collider node (**B**) is **NOT** observed, the path is blocked. If B or any of its descendants **IS** observed, the path becomes active.

Idea (Explaining Away): If the Alarm is ON (observed B), learning about an Earthquake makes a Burglary seem less likely (it "explains away" the alarm).

⭐ Quick Table

Structure	Pattern	When BLOCKED?	When ACTIVE?
Chain	A → B → C	B observed	B not observed
Fork	A ← B → C	B observed	B not observed
Collider	A → B ← C	B not observed	B (or descendant) observed

⚠️ **Collider is the opposite** of the other two.



Chain Structure

Pattern: $X \to Y \to Z$

1

Without Evidence on Y

Dependence flows from X to Z through Y

2

Conditioning on Y

Blocks the path

$$X \perp Z \mid Y$$



Fork Structure

Pattern: $X \leftarrow Y \rightarrow Z$

Y is the **common cause** of both X and Z

Without Evidence

X and Z are dependent through their common cause Y

Conditioning on Y

Blocks dependence:

$$X \perp Z \mid Y$$



Collider Structure

Pattern: $X \rightarrow Y \leftarrow Z$

Y is a **common effect** of X and Z

Unintuitive behavior!

X and Z are independent: $X \perp Z$

BUT conditioning on Y **creates** dependence:

$$X \not\perp Z \mid Y$$

This is called explaining away

Collider: Explaining Away



Burglar Scenario

Alarm sounds. Could be burglary or earthquake.



Learn: Earthquake!

Observing earthquake **explains away** the alarm



Result

Probability of burglary **decreases** given earthquake evidence

d-Separation: Formal Definition

A path is **d-separated** (blocked) by evidence set Z if:

01

Chain or Fork

Path blocked if middle node **is in Z**

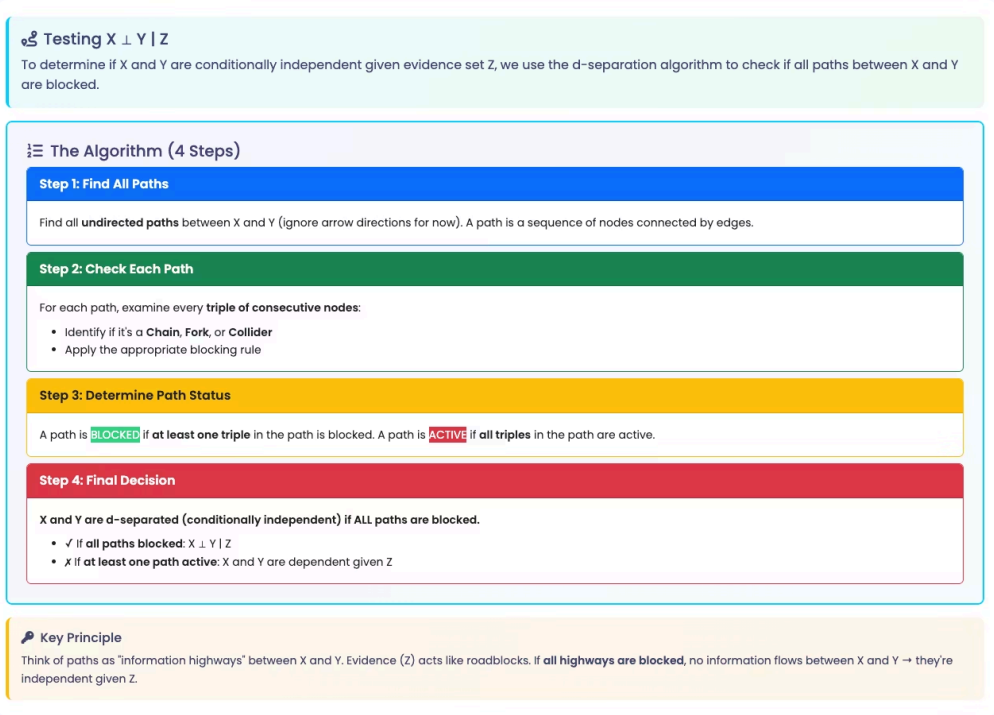
02

Collider

Path blocked **unless** collider or descendant **is in Z**

$$X \perp Y \mid Z \text{ if all paths are d-separated}$$

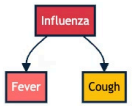
d-Separation Algorithm



Example

Example 2: Fork with No Evidence

Question: Is Fever ⊥ Cough | {} (no evidence)?



Step-by-Step Solution:

Step 1: Find Paths
Path: Fever ← Influenza → Cough (only one path)

Step 2: Identify Structure
Triple: Fever ← Influenza → Cough
This is a **FORK** (A ← B → C)

Step 3: Apply Rule
Fork rule: Blocked if common parent (Influenza) is observed
Evidence set Z = {} → Influenza is NOT observed ✗
Result: Path is ACTIVE

Step 4: Conclusion
At least one path (the only one) is active

✗ NO: Fever and Cough are DEPENDENT (not independent)

Interpretation: Without knowing if the patient has influenza, fever and cough are correlated — observing fever makes cough more likely because both are caused by the common disease.



d-Separation Is Fundamental

d-Separation

For Modeling

- Verify independence assumptions
- Design network structure
- Validate domain knowledge

For Inference

- Identify relevant variables
- Prune search space
- Optimize algorithms

⭐How the d-Separation Algorithm Works

To check if **X ⊥ Y | Z**:

1. **List all paths** between X and Y (ignore arrow directions).
2. For each path, check each triple:
   - Is it a **Chain**?
   - A **Fork**?
   - A **Collider**?
3. Apply the **blocking rule** for that type.
4. If **every path is blocked → Independent**
   If **any path is active → Dependent**

⭐ The Whole Idea in One Sentence

> **If every possible path from X to Y is blocked by the evidence Z, then X and Y are conditionally independent.**

🛠️ Constructing a Bayesian Network

Building a Bayesian Network is a systematic process that transforms a real-world problem into a structured probabilistic model.

Identify Variables

List all relevant **random variables** in the domain, capturing all uncertain quantities.

Determine Dependencies

Draw arrows for **direct influences** (cause → effect) to form the **DAG** (Directed Acyclic Graph).

Order Variables

An optional, but helpful, step: order variables consistent with causality (causes early, effects later).

Fill CPTs

Specify **P(X)** for root nodes and **P(X | Parents(X))** for others. Ensure each row sums to 1.0.

Validate Model

Verify dependencies, missing arrows, CPT accuracy, and ensure the DAG remains acyclic.

Use for Inference

Once built, the BN can answer predictive, diagnostic, and interventional queries.

🌟 Summary: A Bayesian Network = Variables + Structure + CPTs